BACKEND IMPLEMENTATION: CENTRALIZING PROVIDER LOGIC

First, install the necessary provider packages:

```
$ npm install @ai-sdk/openai @ai-sdk/google
```

Next, we centralize the logic for selecting and initializing a model in a single function, `getSupportedModel`. This function acts as a control center, verifying that the requested provider and model are supported and that the corresponding API key exists before returning the correct model instance. The following listing provides the implementation of the `getSupportedModel` function.

Listing 3.11 Function to get supported models

```
import { createOpenAI } from '@ai-sdk/openai';
import { createGoogleGenerativeAI } from '@ai-sdk/google';

const supportedProviders = {.
  openai: {
    constructor: createOpenAI,
    models: ['gpt-3.5-turbo', 'gpt-4'],
  },
  gemini: {
    constructor: createGoogleGenerativeAI,
    models: ['models/gemini-2.0-flash'],
  },
};

export function getSupportedModel(provider, model) {
  const providerConfig = supportedProviders[provider];

  if (!providerConfig) {.
    throw new Error(`Unsupported provider: ${provider}`);
  }

  const { constructor, models } = providerConfig;

  if (!models.includes(model)) {.
    throw new Error(`Unsupported model: ${model} for provider: ${provider}`);
  }

  const apiKey =
            process.env[`${provider.toUpperCase()}_API_KEY`];
  if (!apiKey) {
    throw new Error(`Missing API key for provider: ${provider}`);
  }

  const providerInstance = constructor({ apiKey });
  return providerInstance(model);
}
```

Defines the list of supported providers and models

Verifies if the provider is supported

Verifies if the model is supported

Gets the API key from the environment

Calls the constructor function that returns an instance of the AI model that implements the language model specification